

# TESTE TÉCNICO

## Desenvolvedor Full-Stack Júnior

### *Mini CRM de Leads*

#### Objetivo do desafio

Construir uma aplicação full-stack funcional, organizada e bem documentada, demonstrando domínio básico de frontend, backend, banco de dados, autenticação, arquitetura e uso responsável de IA.

| Item            | Descrição                        |
|-----------------|----------------------------------|
| Prazo máximo    | Enviar até 12/06/26 – Hora 20:00 |
| Perfil avaliado | Desenvolvedor Full-Stack Júnior  |
| Frontend        | Next.js + TypeScript             |
| Backend         | Node.js + Express + TypeScript   |
| Banco/ORM       | PostgreSQL + Prisma              |
| Uso de IA       | Permitido, desde que documentado |

**Entrega:** link do repositório GitHub (Repositório privado) + instruções para rodar localmente no README do repositório. Ao finalizar o teste, enviar convite para “rodrigoamb” para fazer parte do repositório e ter acesso ao Código

**Atenção!!!**

**Não consegui finalizar, o que eu faço?**

**Resposta:** Entregue o que conseguir fazer, leia o documento abaixo com atenção. Boa sorte!

# 1. Contexto

Você deverá desenvolver uma aplicação full-stack chamada Mini CRM de Leads, com frontend em Next.js, backend em Node.js + Express, banco de dados PostgreSQL e ORM Prisma.

O objetivo do teste é avaliar sua capacidade de construir uma aplicação funcional, organizada, bem documentada e com boa separação de responsabilidades.

## Uso de IA permitido

O uso de IA é permitido. Porém, deverá ser documentado de forma transparente na pasta /docs/ai, conforme as instruções deste documento.

# 2. Tecnologias obrigatórias

| Área           | Tecnologias                                                          |
|----------------|----------------------------------------------------------------------|
| Frontend       | Next.js, TypeScript, TailwindCSS ou CSS Modules, consumo de API REST |
| Backend        | Node.js, Express, TypeScript, Prisma ORM                             |
| Banco de dados | PostgreSQL, migrations com Prisma                                    |
| Documentação   | README principal e documentação de IA em Markdown                    |

# 3. Objetivo do projeto

Criar um Mini CRM de Leads onde o usuário autenticado consiga cadastrar leads, organizar esses leads em etapas de um funil e registrar interações com cada lead.

# 4. Funcionalidades obrigatórias

## 4.1 Autenticação

- Cadastro de usuário
- Login
- Logout
- Proteção das rotas privadas
- Usuário autenticado deve visualizar apenas os próprios dados
- Boas práticas de autenticação. (JWT)

## Importante

Não é obrigatório implementar refresh token, recuperação de senha ou confirmação de e-mail.

## 4.2 Cadastro de leads

- Criar lead

- Listar leads
- Editar lead
- Excluir lead
- Visualizar detalhes de um lead

| Campo mínimo do lead | Observação                                      |
|----------------------|-------------------------------------------------|
| Nome                 | Obrigatório                                     |
| Telefone             | Texto livre                                     |
| E-mail               | Formato válido quando informado                 |
| Empresa ou origem    | Ex.: Instagram, indicação, site, tráfego pago   |
| Status no funil      | Novo, Em atendimento, Proposta enviada, Fechado |
| Observação           | Texto livre                                     |
| Data de criação      | Gerada automaticamente                          |

### 4.3 Funil Kanban

Criar uma visualização em formato Kanban com pelo menos 4 colunas:

- Novo
- Em atendimento
- Proposta enviada
- Fechado

A mudança de status pode ser feita por drag and drop ou por botão/select. Drag and drop será considerado diferencial, mas não é obrigatório.

### 4.4 Interações com o lead

Dentro da página de detalhes do lead, o usuário deve conseguir registrar interações, como:

- Ligação
- WhatsApp
- E-mail
- Reunião
- Observação interna

| Campo mínimo da interação | Descrição                                        |
|---------------------------|--------------------------------------------------|
| Tipo                      | Ligação, WhatsApp, E-mail, Reunião ou Observação |
| Descrição                 | Resumo da interação                              |
| Data da interação         | Pode ser gerada automaticamente                  |

### 4.5 Dashboard simples

- Total de leads
- Total de leads por status
- Total de interações registradas

- Últimos leads cadastrados

Gráficos são opcionais.

## 5. Requisitos de arquitetura

### 5.1 Backend

- Espera-se separação e organização dos arquivos e pastas
- Sua capacidade de organizar seu projeto e arquitetura backend será também avaliado aqui

### 5.2 Frontend

- Espera-se separação e organização dos arquivos e pastas
- Sua capacidade de organizar seu projeto e arquitetura backend será também avaliado aqui

## 6. Validações esperadas

- E-mail obrigatório e válido no cadastro/login
- Senha obrigatória
- Nome do lead obrigatório
- Status válido para o funil
- Lead pertencente ao usuário autenticado

Pode ser utilizado Zod, Joi, Yup ou validação manual.

## 8. Segurança mínima esperada

- Senha salva com hash
- JWT ou sessão para autenticação
- Rotas privadas protegidas
- Usuário não pode acessar leads de outro usuário
- Variáveis sensíveis em .env

## 9. Diferenciais

- Docker Compose com PostgreSQL
- Seed de dados
- Testes automatizados
- Responsividade
- Drag and drop no Kanban
- Paginação ou filtros
- Busca por leads
- Tratamento visual de loading e erro
- Documentação clara
- Deploy do frontend e backend
- Organização de commits
- Uso bem documentado de IA (**OBRIGATÓRIO**)

## 10. Uso de IA

O uso de ferramentas de IA é permitido, desde que seja documentado.

Exemplos: ChatGPT, Claude, Gemini, Copilot, Cursor, Windsurf, Codeium ou outros assistentes.

### Regra de transparência

Todo uso relevante de IA deve ser documentado. O objetivo não é punir o uso de IA, mas avaliar se o candidato sabe dar contexto, revisar, adaptar e tomar decisões técnicas.

Criar a seguinte pasta no projeto:

```
/docs/ai
```

Dentro dela, devem existir os seguintes arquivos:

```
/docs/ai/README.md  
/docs/ai/prompts.md  
/docs/ai/decisions.md  
/docs/ai/review.md
```

### 10.1 Arquivo /docs/ai/README.md

- Quais ferramentas de IA foram usadas
- Em quais partes do projeto a IA ajudou
- Em quais partes o candidato escreveu ou decidiu manualmente
- Quais limitações ou erros a IA apresentou
- Como o candidato revisou o código gerado

### 10.2 Arquivo /docs/ai/prompts.md

Deve conter os principais prompts usados durante o desenvolvimento. Não precisa registrar conversas inteiras, mas deve registrar os prompts mais importantes.

```
# Prompts utilizados  
  
## Prompt 1 - Estrutura inicial do backend  
Ferramenta usada: ChatGPT  
  
Prompt:  
"Me ajude a estruturar uma API Express com TypeScript, Prisma, PostgreSQL, autenticação JWT e separação por routes, controllers e services."  
  
Resultado:  
A IA sugeriu uma estrutura inicial de pastas. Eu adaptei a organização para o padrão que escolhi no projeto.  
  
O que aproveitei:  
- Estrutura de pastas  
- Ideia inicial dos middlewares
```

O que alterei:

- Nome dos arquivos
- Regra de autenticação
- Organização dos services

### 10.3 Arquivo /docs/ai/decisions.md

Deve explicar **COM SUAS PALAVRAS** as principais decisões técnicas do projeto, por exemplo:

- Por que escolheu JWT
- Como organizou as camadas do backend
- Como protegeu os dados por usuário
- Como estruturou o Prisma
- Como organizou chamadas de API no frontend
- Como tratou erros
- Como validou dados

### 10.4 Arquivo /docs/ai/review.md

- O que foi gerado por IA?
- O que foi revisado manualmente?
- Houve algum erro da IA?
- Algum código foi descartado?
- Quais melhorias foram feitas depois da geração inicial?

## 11. Entrega

1. Link do repositório no GitHub (Repo privado e convidar rodrigoamb para o repo)
2. Instruções para rodar o projeto localmente
3. Arquivo .env.example
4. Migrations do Prisma
5. Documentação do uso de IA na pasta /docs/ai
6. Opcional: link do deploy
7. Commits semânticos e organizados

## 12. README principal

O arquivo README.md na raiz do projeto deve explicar:

- Descrição do projeto
- Tecnologias utilizadas
- Como instalar
- Como configurar variáveis de ambiente
- Como rodar migrations
- Como rodar backend
- Como rodar frontend
- Usuário de teste, se houver seed

- Funcionalidades entregues
- Funcionalidades que ficaram pendentes
- Decisões técnicas principais

## 13. O que será observado no uso de IA

| Ponto avaliado                | O que observar                                                                     |
|-------------------------------|------------------------------------------------------------------------------------|
| Transparência                 | O candidato documentou prompts, ferramentas e trechos gerados ou revisados com IA? |
| Qualidade dos prompts         | O candidato deu contexto, delimitou escopo e orientou a IA com clareza?            |
| Capacidade de revisão         | O candidato entendeu, corrigiu e adaptou o que a IA entregou?                      |
| Orquestração por arquivos .md | O candidato usou documentação para organizar contexto, decisões e revisão?         |

## 15. Observação final

**BOA SORTE!!**

### Mensagem ao candidato

O objetivo deste teste não é avaliar apenas se você consegue entregar muitas funcionalidades. O objetivo principal é entender como você pensa, organiza o código, resolve problemas, documenta decisões e utiliza ferramentas modernas, incluindo IA, de forma responsável.